PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UE24CS351A: Software Engineering Laboratory — Lab 5

Student Name:	Neel Chandrakar	**SRN:**	PES2UG24CS310
Course / Section:	Software Engineering Lab (Sec F)	**Submission Date:**	October 14, 2026
Target Program:	energy_system.py	**Assigned Repo:**	SETAPESU26/L5_03_energy_tariff

1. Objective & Static Analysis Results

The objective of this laboratory exercise is to systematically analyze, detect, document, and rectify code quality flaws, security vulnerabilities, and PEP 8 style violations in energy_system.py. Using **Pylint**, **Bandit**, and **Flake8**, all flagged defects were resolved to achieve a perfect **10.00/10 Pylint score**, **0 Bandit vulnerabilities**, and **0 Flake8 violations**.

Static Analysis Tool	Baseline Metric (Pre-Fix)	Final Metric (Post-Fix)	Status
Pylint (Quality / AST)	6.14 / 10.00	**10.00 / 10.00 (+3.86)**	**PERFECT**
Bandit (Security / CWE)	1 Vulnerability (B307 eval)	**0 Vulnerabilities (Clean)**	**RESOLVED**
Flake8 (PEP 8 Style)	12 Violations (E501, E302, E722)	**0 Violations (Clean)**	**RESOLVED**

2. Documented Issues & Remediation Table

All issues identified across Pylint, Bandit, and Flake8 are cataloged below with severity, exact line references, and remediation strategies:

Issue	Type	Line	Problem Description	Fix Approach
Insecure eval() Call	Security (Bandit B307 / CWE-78)	63	User input passed directly into eval(), allowing Remote Code Execution (RCE).	Replaced with safe numeric parsing (safe_parse_expression) with exception guards.
Mutable Default Argument	Bug (Pylint W0102)	12	history=[] evaluated once at definition, leaking reading state across calls.	Defaulted history to None; initialized fresh list inside body when None.
Bare except: Clause	Bug / Smell (Pylint W0702, Flake8 E722)	44	Catches BaseException, masking KeyboardInterrupt, SystemExit, and bugs.	Caught explicit exceptions: (FileNotFoundError, json.JSONDecodeError, OSError).
Unmanaged File Handles	Resource Leak (Pylint R1732, W1514)	42, 48	open() called without context managers (with) or explicit UTF-8 encoding.	Wrapped file I/O in with open(..., encoding='utf-8') context managers.
Unused Module Import	Code Smell (Pylint W0611, Flake8 F401)	2	import sys was imported at module level but never referenced anywhere.	Removed unused sys import statement completely.
Global Variable Mutation	Architecture (Pylint W0603)	27, 40	global statement used to mutate module-level variables inside functions.	Encapsulated state into structured SYSTEM_STATE container dict.
Legacy % Formatting	Style (Pylint C0209)	59, 60, 66	printf-style % interpolation used instead of modern PEP 498 f-strings.	Converted all string formatting to clean, readable Python 3.6+ f-strings.
Missing Docstrings	Documentation (Pylint C0114, C0116)	1, 12, 17, 26, 32, 39, 47, 52	Module and functions lacked PEP 257 docstrings defining args and return types.	Added comprehensive docstrings with argument and return specifications.
PEP 8 Style & Spacing	Style (Flake8 E302, E305, E501)	Multiple	Lines exceeded 79 characters (88 chars); 1 blank line found instead of 2.	Formatted top-level spacing with 2 blank lines and wrapped lines <= 79 chars.
Negative Reading Domain Bug	Domain Logic Bug	14, 55	Permitted negative consumption, generating negative electricity bills.	Added validation in record_reading to flag negative readings.

3. Answers to Reflection Questions

Question 1: What is the purpose and scope of static code analysis in the SDLC?
Static code analysis examines source code without execution to identify security vulnerabilities, logical bugs, and style non-compliance early in the software development lifecycle ('shift-left' testing). Unlike dynamic testing, which relies on executable test cases and runtime coverage, static analysis inspects 100% of reachable code paths instantly. It serves as an automated code reviewer, reducing defect remediation costs by catching problems prior to staging and production. However, it cannot validate whether business logic correctly meets domain specifications, requiring complementary functional testing.

Question 2: How do Pylint, Bandit, and Flake8 differ in scope and mechanics?
Each tool specializes in a distinct quality dimension:
• **Pylint**: Deep AST analyzer evaluating code design, logical bugs (e.g., mutable defaults, unreachable code), and code smells. It enforces modularity and computes a quantitative quality score (rated 10.00/10 upon remediation).
• **Bandit**: AST-driven security scanner checking code against Common Weakness Enumeration (CWE) vulnerability databases. It identified the unsafe eval() expression on Line 63 as a high-confidence security risk.
• **Flake8**: Combines PyFlakes, pycodestyle, and McCabe complexity to enforce PEP 8 style, line length limits (≤79 chars), and clean import conventions.
Using all three in concert provides holistic coverage across syntax, architecture, and security.

Question 3: What specific issues were found in energy_system.py and why were they dangerous?
• *Insecure eval() (Line 63)*: Permitted arbitrary string execution, exposing the host system to Remote Code Execution (RCE). Resolved by replacing eval() with a safe numeric expression parser (safe_parse_expression).
• *Mutable Default List (Line 12)*: history=[] caused a single list to be shared across all invocations, leaking state across independent calls. Resolved using the None guard idiom.
• *Bare except (Line 44)*: Caught all exceptions including system signals (KeyboardInterrupt), masking errors during debugging. Replaced with explicit exception tuples (FileNotFoundError, JSONDecodeError, OSError).
• *Unclosed File Handles (Lines 42, 48)*: Raw open() calls without context managers risked file descriptor leaks. Wrapped with with open(...) context managers.
• *Domain Logic Bug*: Negative units (-50) caused negative electricity billing (-192.50), addressed via validation.

Question 4: What is the long-term impact of static analysis on software maintainability and security?
Continuous static code analysis integrated into CI/CD pipelines and pre-commit hooks eliminates technical debt accumulation, prevents recurring security flaws from reaching production, and enforces consistent style across distributed teams. It reduces code review overhead by automating mechanical checks and guarantees self-documenting code through docstring enforcement.

4. Post-Remediation Verification Tool Outputs

Pylint 4.1.3 Verification Output (Score: 10.00 / 10.00)

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 6.14/10, +3.86)
```

Bandit 1.9.4 Security Scan Verification Output

```
Run started: 2026-10-14 14:21:04 UTC
Test results: No issues identified.
Total lines of code: 151 | Total issues: 0 (High: 0, Medium: 0, Low: 0)
```

Flake8 7.4.1 PEP 8 Compliance Verification Output

```
flake8 energy_system.py -> Clean (0 errors / 0 warnings returned with exit code 0)
```